Reflected XSS into a Template Literal with Angle Brackets, Single, Double Quotes, Backslash and Backticks Unicode-Escaped

1. Overview

Field	Details
Vulnerability	Reflected Cross-Site Scripting (XSS)
Category	Client-Side Injection
Injection Context	JavaScript Template Literal
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy
Vulnerability Type	Reflected XSS

2. Description

Reflected Cross-Site Scripting occurs when an application includes user-controlled input in an HTTP response without properly neutralizing it for the output context.

In this lab, the application reflects user input inside a JavaScript template literal.

The application Unicode-escapes several potentially dangerous characters, including angle brackets, single quotes, double quotes, backslashes, and backticks. However, character escaping alone does not guarantee that the resulting JavaScript remains safe.

JavaScript template literals support expression interpolation through `${...}` syntax. If untrusted input can introduce an expression into this context, it may be interpreted and executed by the browser.

3. Root Cause

The underlying issue is unsafe insertion of user-controlled data into an executable JavaScript template literal.

The application relies on escaping a predefined set of characters instead of ensuring that untrusted input cannot alter the JavaScript parsing structure.

Template literal interpolation introduces a separate parsing mechanism that must be considered when securing this context.

4. Proof of Concept (PoC)

Testing environment: Authorized PortSwigger Web Security Academy lab.

1. Open the assigned lab.
2. Identify the input parameter reflected inside a JavaScript template literal.
3. Submit a harmless test value.
4. Inspect the HTTP response and locate the template literal.
5. Examine how special characters are Unicode-escaped.
6. Determine whether template literal expression interpolation is possible.
7. In the authorized lab, test a controlled expression that demonstrates whether injected input is evaluated.
8. Confirm the behavior using browser developer tools.

Example vulnerable context:

```
None
const searchTerm = `USER_INPUT`;
```

Template literals also support expressions such as:

```
None
const message = `Result: ${1 + 1}`;
```

These examples illustrate the parsing context and are not confirmed working lab payloads.

5. Security Impact

Successful exploitation may allow an attacker to:

- Execute arbitrary JavaScript in the vulnerable website's origin.
- Manipulate webpage content and application behavior.
- Perform unauthorized actions through a victim's authenticated session.

- Access information available to client-side scripts.
- Redirect victims to malicious websites.

The actual impact depends on the application's functionality and security controls.

6. Recommended Remediation

- Avoid inserting untrusted input into executable JavaScript template literals.
- Keep application data separate from executable code.
- Use safe JSON serialization when transferring data to client-side JavaScript.
- Apply context-aware output encoding.
- Use secure templating mechanisms that prevent JavaScript injection.
- Avoid relying exclusively on character-based blacklists or Unicode escaping.
- Implement a restrictive Content Security Policy (CSP).
- Add regression tests for template literal interpolation and special-character handling.

7. Verification

After remediation:

1. Repeat the original lab test cases.
2. Confirm that user-controlled input cannot introduce executable template literal expressions.
3. Verify that special characters remain data.
4. Test different encoding variations.
5. Confirm normal application functionality.

8. References

- PortSwigger: Cross-Site Scripting
- PortSwigger: XSS Contexts
- MDN: Template Literals
- OWASP: XSS Prevention Cheat Sheet

9. Disclaimer

This report is intended for educational purposes and authorized security testing only.